Лабораторная работа
«JavaScript, фильтр и сортировка»

Задание 1. Задание. Создать HTML-страницу, на которую вывести таблицу (рисунок 1). Таблица строится на основе массива **`buildings`** следующей структуры (весь массив приведен в файле **`data.js`**, прикреплен к Лабораторной работе):

```
const buildings = [
    {
        "Название": "Бурдж-Халифа",
        "Тип": "Небоскрёб",
        "Страна": "ОАЭ",
        "Город": "Дубай",
        "Год": 2010,
        "Высота": 828
    },
…
]
```

Название	Тип	Страна	Город	Год	Высота
Бурдж-Халифа	Небоскрёб	ОАЭ	Дубай	2010	828
Варшавская радиомачта	Антенная мачта	Польша	Константинов	1974	646.38
Tokyo Skytree	Бетонная башня	Япония	Токио	2012	634
Шанхайская башня	Небоскрёб	КНР	Шанхай	2013	632
Телерадиомачта KVLY-TV	Радиомачта	США	Бланчард	1963	629
Телебашня Гуанчжоу	Гиперболоидная башня	КНР	Гуанчжоу	2009	600
Международный финансовый центр Пинань	Небоскрёб	КНР	Шэньчжэнь	2017	600

Рисунок 1 – HTML-страница с таблицей

Порядок выполнения практического задания

1. Создать папку **`practicum_JS`** следующей структуры:

```
practicum_JS
|——— CSS
|      |——— style.css
|——— JavaScript
|      |——— data.js /* вставить файл с данными */
|      |——— main.js /* главная программа */
|      |——— table.js /* создание таблицы и вывод таблицы */
|——— index.html
```

2. Создать файл **`index`**.

```
<html lang="ru">
<!DOCTYPE html>
<html lang="ru">
  <head>
    <meta charset="utf-8">
    <title>Фильтры и сортировка</title>
    <link rel="stylesheet" href="CSS/style.css">
    <script src="JavaScript/data.js"></script>
    <script src="JavaScript/main.js"></script>
    <script src="JavaScript/table.js"></script>
```

```
  </head>
  <body>
    <table id="list">
    </table>
  </body>
</html>
```

3. Создать файл **style.css**, в который добавить стили так, чтобы страница выглядела, как показано на рисунке 1 (оформление таблицы можно придумать самостоятельно).

4. Создать файл **table.js**, в нем реализовать функцию **createTable()** для вывода таблицы на страницу. Функция в качестве параметра получает массив для формирования таблицы и имя таблицы на странице. В функции создается шапка таблицы (функция **createHeaderRow()**).

```
const createTable = (data, idTable) => {
    const table = document.getElementById(idTable);
    const header = Object.keys(data[0]);

    /* создание шапки таблицы */
    const headerRow = createHeaderRow(header);
    table.append(headerRow);

    /* создание тела таблицы */
    /* const bodyRows = createBodyRows(data);
    table.append(bodyRows); */
};

const createHeaderRow = (headers) => {
    const tr = document.createElement('tr');
    headers.forEach(header => {
        const th = document.createElement('th');
        th.innerHTML = header;
        tr.append(th);
    });
    return tr;
};
```

5. В файл **main.js** добавить обработчик события, в котором формируется таблица при загрузке страницы.

```
document.addEventListener("DOMContentLoaded", function() {

    createTable(buildings, 'list');

})
```

В результате после загрузки на странице отобразится шапка таблицы.

6. В функции **createTable()** раскрыть комментарий, в котором формируется тело таблицы. **Самостоятельно** реализовать функцию **createBodyRows()** по следующему алгоритму:

1. создать элемент **tbody**;
2. перебрать все элементы массива **data**;
3. для каждого элемента массива:
   - создать новую строку таблицы **tr**;

- для каждой пары ключ - значение:
  - создать элемент **`td`**;
  - занести в него значение;
  - добавить элемент **`td`** к строке.
- строку добавить к элементу **`tbody`**.

4. Вернуть из функции сформированный **`tbody`**.

После обновления страницы **`index.html`** в браузере, страница должна выглядеть, как показано на рисунке 1.

Задание 2

Добавить на HTML-страницу форму с фильтрами (рисунок 2) и реализовать фильтрацию таблицы.

Пользователь в текстовые поля вводит текст, в соответствующем столбце должен осуществляться поиск на вхождение введенного текста в любом месте содержимого.

Для фильтрации числовых значений используется диапазон. При этом считается, если первое поле диапазона не заполнено, то там занесено значение минус бесконечность, если же второе поле диапазона не заполнено, считается, что занесено значение плюс бесконечность.

При клике по кнопке **Найти** на страницу из выводятся только те строки, которые отвечают всем фильтрам одновременно.

При клике на кнопку **Очистить фильтры** – все поля формы очищаются и на страницу выводится исходная таблица.

Фильтр

Название:

Тип:

Страна:

Город:

Год: от до

Высота: от до

Найти | Очистить фильтры

Название	Тип	Страна	Город	Год	Высота
Бурдж-Халифа	Небоскрёб	ОАЭ	Дубай	2010	828
Варшавская радиомачта	Антенная мачта	Польша	Константинов	1974	646.38
Tokyo Skytree	Бетонная башня	Япония	Токио	2012	634
Шанхайская башня	Небоскрёб	КНР	Шанхай	2013	632
Телерадиомачта KVLY-TV	Радиомачта	США	Бланчард	1963	629
Телебашня Гуанчжоу	Гиперболоидная башня	КНР	Гуанчжоу	2009	600
Международный финансовый центр Пинань	Небоскрёб	КНР	Шэньчжэнь	2017	600

Рисунок 2 – HTML-страница с таблицей и формой с фильтрами

Порядок выполнения практического задания

1. В файл **index.html** добавить форму с полями для фильтрации:

```html
<form id="filter">
  <p>
    <label for="structure">Название:</label>
    <input type="text" id="structure">
  </p>
  <p>
    <label for="category">Тип:</label>
    <input type="text" id="category">
  </p>
  <p>
    <label for="country">Страна:</label>
    <input type="text" id="country">
  </p>
  <p>
    <label for="city">Город:</label>
    <input type="text" id="city">
  </p>
  <p>
    <label for="yearFrom">Год: </label>
    от <input type="number" id="yearFrom">
    до <input type="number" id="yearTo">
  </p>
  <p>
    <label for="heightFrom">Высота:</label>
    от <input type="number" id="heightFrom">
    до <input type="number" id="heightTo">
  </p>
  <input type="button" value="Найти">
  <input type="button" value="Очистить фильтры>
</form>
```

2. Самостоятельно в файле **table.js** создать функцию, которая "очищает" таблицу с **id**, переданным в качестве параметра (то есть удаляет все ее строки).

```
const clearTable = (idTable) => {
 ...
}
```

3. В папке **JavaScript** создать файл **filter.js**. В файле **index.js** добавить соответствующий **script**.

4. В файл **filter.js** добавить ассоциативный массив, в котором будет указано, для фильтрации какого столбца таблицы какие поля формы применяются:

```
// устанавливаем соответствие между полями формы и столбцами таблицы
const correspond = {
    "Название": "structure",
    "Тип": "category",
    "Страна": "country",
    "Город": "city",
    "Год": ["yearFrom", "yearTo"],
    "Высота": ["heightFrom", "heightTo"]
}
```

3. В файле **filter.js** создать функцию **dataFilter()**, которая в качестве параметра получает элементы формы для фильтрации. Функция возвращает ассоциативный массив, в котором хранятся **id** полей и значения, занесенные в поля.

```
/* Структура возвращаемого ассоциативного массива:
{
    input_id: input_value,
    ...
}
*/
const dataFilter = (dataForm) => {

    let dictFilter = {};

    // перебираем все элементы формы с фильтрами
    for (const item of dataForm.elements) {

        // получаем значение элемента
        let valInput = item.value;

        // если поле типа text - приводим его значение к нижнему регистру
        if (item.type === "text") {
            valInput = valInput.toLowerCase();
        }
        /* САМОСТОЯТЕЛЬНО обработать значения числовых полей:
        - если в поле занесено значение - преобразовать valInput к числу;
        - если поле пусто и его id включает From  - занести в valInput
           -бесконечность
        - если поле пусто и его id включает To  - занести в valInput
           +бесконечность
        */

         // формируем очередной элемент ассоциативного массива
        dictFilter[item.id] = valInput;
    }
    return dictFilter;
}
```

4. В файле **filter.js** создать функцию **filterTable()** для фильтрации таблицы, которая в качестве параметров получает исходный массив для формирования таблицы, имя таблицы на странице и данные формы с полями для фильтрации. В функции отбираются те строки массива, которые удовлетворяют всем фильтрам одновременно.

```
const filterTable = (data, idTable, dataForm) =>{

    // получаем данные из полей формы
    const datafilter = dataFilter(dataForm);

    // выбираем данные соответствующие фильтру и формируем таблицу из них
    let tableFilter = data.filter(item => {

        /* в этой переменной будут "накапливаться" результаты сравнения данных
           с параметрами фильтра */
        let result = true;

        // строка соответствует фильтру, если сравнение всех значения из input
        // со значением ячейки очередной строки - истина
         Object.entries(item).map(([key, val]) => {

            // текстовые поля проверяем на вхождение
            if (typeof val == 'string') {
```

```
                result &&= val.toLowerCase().includes(datafilter[correspond[key]])
            }

            // САМОСТОЯТЕЛЬНО проверить числовые поля на принадлежность интервалу
        });

        return result;
    });

    // САМОСТОЯТЕЛЬНО вызвать функцию, которая очищает таблицу с id=idTable

    // показать на странице таблицу с отфильтрованными строками
    createTable(tableFilter, idTable);
}
```

5. Настроить обработку события **`click`** по кнопке **Найти** (вызвать функцию **`filterTable()`** с соответствующими параметрами). Обработчик события НЕ ВКЛЮЧАТЬ в html-код.

В результате, если указать в поле фильтра **Страна** значение «К», в поле **Год от** значение **2010** , в поле **Высота до** значение **450** и кликнуть по кнопке **Найти**, страница будет выглядеть, как показано на рисунке 3.

Самостоятельное задание

1. Обработать ситуацию, если в результате фильтрации не выбрано ни одной записи. В этом случае вывести на страницу только шапку таблицы.

2. Реализовать функцию **`clearFilter()`**, которая будет вызываться по клику по кнопке **Очистить фильтры** (функция в качестве параметра получает **`id`** таблицы, данные для ее построения и данные из формы). Связать вызов этой функции с событием **`click`** по кнопке **Очистить** фильтры. Функция должна очищать все поля формы и выводить таблицу с исходными данными (массив **`buildings`**).

В результате после клика по кнопке **Очистить фильтры** страница будет выглядеть как показано на рисунке 2.

Фильтр

Название:

Тип:

Страна: К

Город:

Год: от 2010 до

Высота: от до 450

Найти | Очистить фильтры

Название	Тип	Страна	Город	Год	Высота
Международный финансовый центр. башня зап.	Небоскрёб	КНР	Гуанчжоу	2010	437.5
Kingkey 100	Небоскрёб	КНР	Шэньчжэнь	2011	439.8
Башня Аль-Хамра	Небоскрёб	Кувейт	Кувейт	2010	412
Абу-Даби Плаза	Небоскрёб	Казахстан	Астана	2015	382

Рисунок 3 – HTML-страница с отфильтрованными данными

Задание 3

Добавить на HTML-страницу форму с настройкой сортировки (рисунок 4). Для этого

1. При загрузке страницы сформировать поля со списком формы для настройки сортировки:

- первым элементом всех списков должна быть опция со значением «Нет»;
- остальные опции – значение ключей элементов массива **`buildings`**;
- при загрузке страницы все поля, кроме поля для сортировки первого уровня, сделать неизменяемыми.

2. При изменении поля для выбора первого уровня сортировки, удалить из списка полей для второго уровня сортировки выбранное в поле первого уровня значение и сделать поле доступным для изменения.

3. При выборе опции «Нет» в поле для первого уровня сортировки, сделать недоступным для изменения поле для второго уровня сортировки.

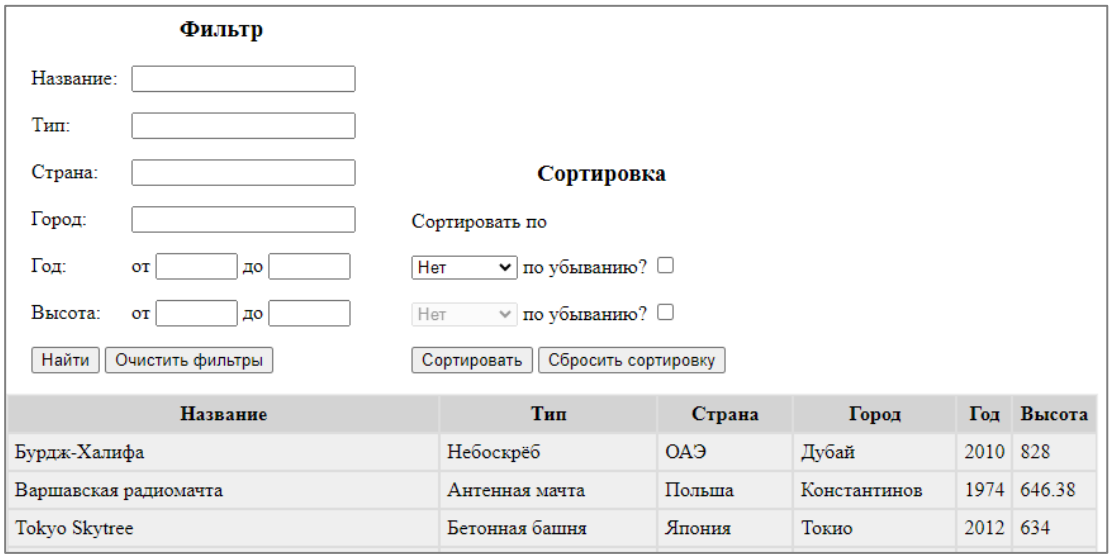

Рисунок 4 – HTML-страница с формой для сортировки

Порядок выполнения практического задания

1. В файл **`index.html`** добавить форму с полями для сортировки:

```
<form id="sort">
  <p> Сортировать по</p>
  <p>
    <select id="fieldsFirst">
    </select>
    по убыванию? <input  type="checkbox"  id="fieldsFirstDesc">
  </p>
  <p>
    <select id="fieldsSecond">
    </select>
    по убыванию? <input  type="checkbox"  id="fieldsSecondDesc">
  </p>
  <input type="button" value="Сортировать">
  <input type="button" value="Сбросить сортировку">
</form>
```

Имена столбцов для управления сортировки формируется как имя соответствующего поля со списком и слова "**Desc**", например, **`fieldsFirstDesc`**.

2. В файле **`main.js`** реализовать функцию **`createOption()`**, которая формирует одну опцию поля со списком (параметры - строка, которую нужно вывести, и значение атрибута **`value`** этой опции).

```
// формирование полей элемента списка с заданным текстом и значением
const createOption = (str, val) => {
    let item = document.createElement('option');
    item.text = str;
    item.value = val;
    return item;
}
```

3. В файле **`main.js`** реализовать функцию **`setSortSelect()`**. Функция в качестве параметров получает массив со значениями, которые нужно занести в элементы **`option`** поля со списком, и элемент select, который нужно сформировать.

```
// формирование поля со списком
// параметры - массив со значениями элементов списка и элемент select

const setSortSelect = (arr, sortSelect) => {

    // создаем OPTION Нет и добавляем ее в SELECT
    sortSelect.append(createOption('Нет', 0));
    // перебираем массив со значениями опций
     arr.forEach((item, index) => {
       // создаем OPTION из очередного ключа и добавляем в SELECT
       // значение атрибута VALUE увеличиваем на 1, так как значение 0 имеет
опция Нет
        sortSelect.append(createOption(item, index + 1));
    });
}
```

4. В файле **`main.js`** реализовать функцию **`setSortSelects`**(), которая в качестве параметра получает первый элемент массива **`buildings`** и данные из формы с настройкой сортировки, а формирует поля со списком для двух уровней сортировки.

```
// формируем поля со списком для многоуровневой сортировки
const setSortSelects = (data, dataForm) => {

    // выделяем ключи словаря в массив
    const head = Object.keys(data);

    // находим все SELECT в форме
    const allSelect = dataForm.getElementsByTagName('select');

    for(const item of dataForm.elements){
        // формируем очередной SELECT
        setSortSelect(head, item);

        // САМОСТОЯТЕЛЬНО все SELECT, кроме первого, сделать неизменяемым
    }
}
```

5. В файле **`main.css`** в обработчике события ***DOMContentLoaded*** добавить вызов функции **`setSortSelects()`** с соответствующими параметрами.

В результате после загрузки страница должна выглядеть, как показано на рисунке 4.

6. Создать функцию **changeNextSelect()** в файле **main.js**. В качестве параметров она получает ссылку на текущий элемент **SELECT** и **id** поля со следующим уровнем сортировки. Функция формирует следующий уровень сортировки без той опции, которая выбрана в текущем поле со списком (если выбрана опция «Нет» – поле становится недоступным).

```
// настраиваем поле для следующего уровня сортировки
const changeNextSelect = (curSelect, nextSelectId) => {

    let nextSelect = document.getElementById(nextSelectId);

    nextSelect.disabled = false;

    // в следующем SELECT выводим те же option, что и в текущем
    nextSelect.innerHTML = curSelect.innerHTML;

    // удаляем в следующем SELECT уже выбранную в текущем опцию
    // если это не первая опция - отсутствие сортировки
    if (curSelect.value != 0) {
       nextSelect.remove(curSelect.value);
    } else {
        nextSelect.disabled = true;
    }
}
```

7. С событием **change** поля для первого уровня сортировки связать функцию **changeNextSelect()**, в качестве параметров передать ссылку на текущий элемент **SELECT** и **id** следующего элемента **SELECT** (**fieldsSecond**).

В результате, если в поле для первого уровня сортировки выбран столбец **Тип**, то в поле для второго уровня этот столбец не отображается (рисунок 5).

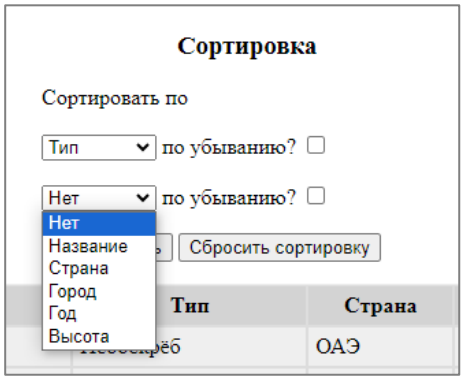

Рисунок 5 – Форма для настройки сортировки

Задание 4

Реализовать двухуровневую сортировку таблицы, выведенной на html-страницу. Способ сортировки каждого уровня (по убыванию или возрастанию) выбирать в зависимости от указанного в форме направления сортировки.

Сортировку таблицы осуществить по клику на кнопку **Сортировать** (рисунок 6).

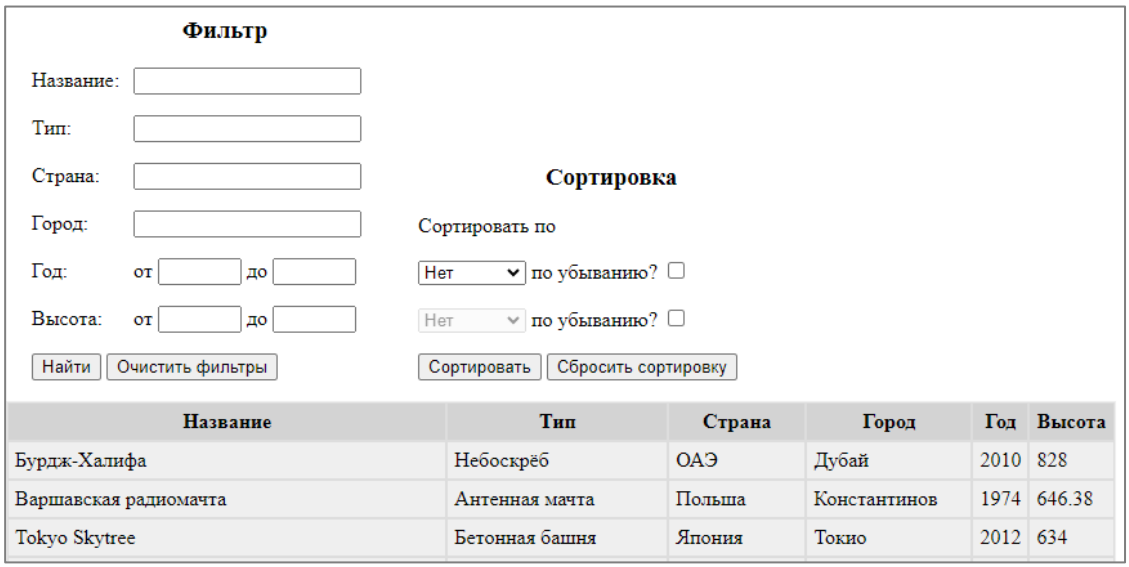

Рисунок 6 – HTML-страница с формой для сортировки

Порядок выполнения практического задания

1. В папке JavaScript создать файл **`sort.js`**. Добавить соответствующий **`script`** в файл **`index.html`**. В этом файле создать функцию **`createSortArr()`**, которая на входе получает данные формы для настройки сортировки. Функция формирует массив, в который включаются те уровни сортировки, в которых не выбрана опция «Нет».

```
/*формируем массив для сортировки по двум уровням вида
  [
    {column: номер столбца, по которому осуществляется сортировка,
     direction: порядок сортировки (true по убыванию, false по возрастанию)
    },
    ...
   ]
*/
const createSortArr = (data) => {
    let sortArr = [];

    const sortSelects = data.getElementsByTagName('select');

    for (const item of sortSelects) {
       // получаем номер выбранной опции
        const keySort = item.value;
        // в случае, если выбрана опция Нет, заканчиваем формировать массив
        if (keySort == 0) {
            break;
        }
        // получаем порядок сортировки очередного уровня
        // имя флажка сформировано как имя поля SELECT и слова Desc
        const desc = document.getElementById(item.id + 'Desc').checked;
```

```
        //очередной элемент массива - по какому столбцу и в каком порядке сортировать
        sortArr.push(
          {column: keySort - 1,
           direction: desc}
        );
    }
    return sortArr;
};
```

2. В файле **sort.js** создать функцию **sortTable()** для сортировки таблицы, выведенной на странице, по возрастанию для всех уровней сортировки. Функция в качестве параметра получает **id** таблицы и данные формы для настройки сортировки. Функция реализует многоуровневую сортировку, каждый уровень сортируется в зависимости от указанного в форме направления сортировки.

```
const sortTable = (idTable, formData) => {

    // формируем управляющий массив для сортировки
    const sortArr = createSortArr(formData);

    // сортировать таблицу не нужно, во всех полях выбрана опция Нет
    if (sortArr.length === 0) {
        return false;
    }
    //находим нужную таблицу
    let table = document.getElementById(idTable);

    // преобразуем строки таблицы в массив
    let rowData = Array.from(table.rows);

    // удаляем элемент с заголовками таблицы
     const headerRow = rowData.shift();

    //сортируем данные по всем уровням сортировки
    rowData.sort((first, second) => {
        for (let { column, direction } of sortArr) {
           const firstCell = first.cells[column].innerHTML;
           const secondCell = second.cells[column].innerHTML;

           // используем localeCompare для корректного сравнения
           const comparison = firstCell.localeCompare(secondCell);

           // учитываем направление сортировки
           if (comparison !== 0) {
             return (direction ? -comparison : comparison);
          }
        }
        return 0;
    });

    //выводим отсортированную таблицу на страницу
    table.append(headerRow);

    let tbody = document.createElement('tbody');
    rowData.forEach(item => {
        tbody.append(item);
    });
    table.append(tbody);
}
```

3. С событием **`click`** по кнопке **Сортировать** связать вызов функции `sortTable()` с соответствующими параметрами.

В результате, если сначала отфильтровать таблицу по названию страны (буква «а» включается) и году (старше 2000). А затем в поле для первого уровня сортировки выбрать столбец **Тип**, в поле для второго уровня выбрать столбец **Год** и кликнуть по кнопке **Сортировать**, страница будет выглядеть, как показано на рисунке 6.

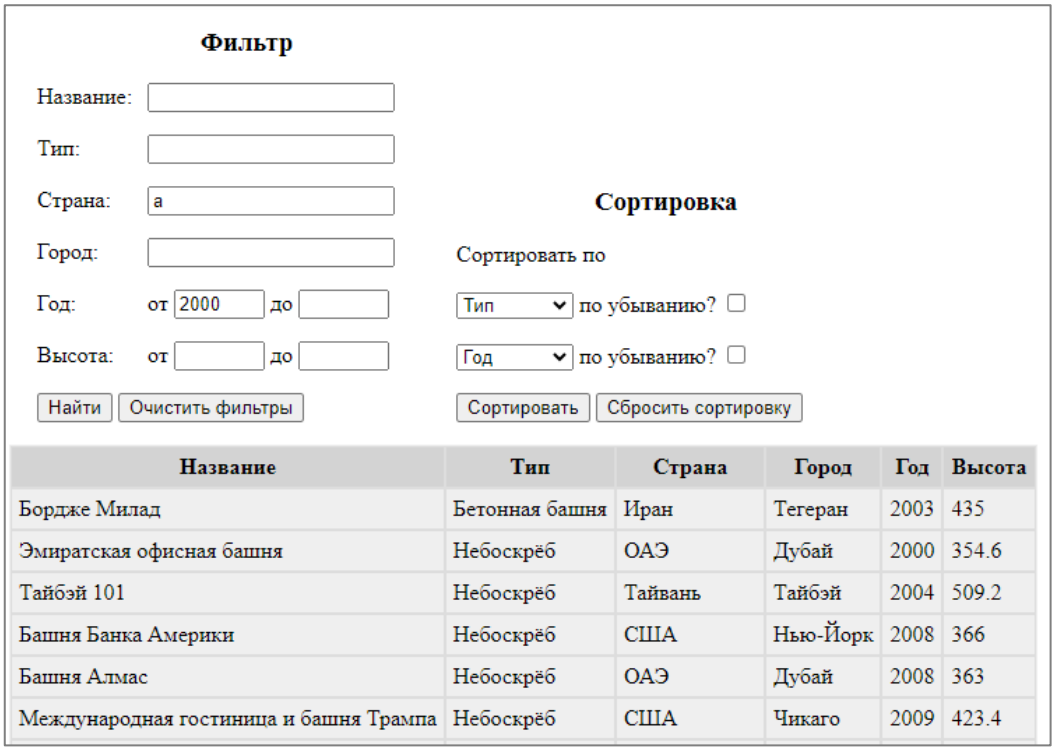

Рисунок 6 – Страница с отсортированной таблицей по возрастанию

Самостоятельное задание

1. Исправить функцию **`sortTable()`** так, чтобы данные в столбцах с числами (**Год** и **Высота**) сортировались как числа, а не как строки.
2. Исправить функцию **`sortTable()`**: если во всех полях сортировки выбрано «**Нет**», восстановить на странице ту таблицу, которая была до применения сортировки.
3. Реализовать функцию для сброса сортировки, которая:
- формирует поля формы для настройки сортировки как при загрузке страницы;
- восстанавливает на странице ту таблицу, которая была до применения сортировки;
- связать эту функцию с событием клик по кнопке **Сбросить сортировку**.
4. В обработчик события поля **Найти** и **Очистить** фильтры добавить сброс всех уровней сортировки в форме (как при загрузке страницы) и вывод данных в таблице в том порядке, в котором они хранятся в массиве **`buildings`**.